When should the pump stay in alarm?

Open the English notebook in Google Colab

If Colab cannot open the link, download this notebook from the course site and use **File → Uplo notebook** in Colab. Save your own copy before editing.

Intro to ML · Time series laboratory · 90 minutes · STUDENT VERSION

One real industrial task, several representations, one frozen held-out evaluation. We compare rc statistics, PCA, Isolation Forest, score fusion and a recorded **Chronos-2 -small** forecasting det SKAB pump measurements.

Required hardware: free Colab CPU, standard RAM. No account token, API key, GPU or found model download is required for the assessed route. Fresh classical fitting runs here. Foundation forecasts are explicitly recorded outputs of the exact causal protocol documented below. An opti extension appears at the end.

Your work: four small code tasks, predictions, failure analysis and one final decision. Prepared t to ML with the instructor and OpenAI Codex. The instructor retains responsibility for accuracy an assessment.

The route

Minutes	Question
0 – 18	What is the fault state, and what information is available now?
18 – 30	Can a normal relationship fail while individual values look plausible?
30 – 45	Does history help the same Isolation Forest?
45 – 57	What does a detector score mean on a different scale?
57 – 68	Does combining three detectors help?
68 – 78	Does forecasting error cover the whole fault state?
78 – 90	Freeze, reveal held-out runs, and defend a decision.

Run this notebook **section by section**. An unsolved function intentionally raises `NotImplementedError`; finish that task before continuing. After completing all four, a fresh to bottom run should work. Supplied plotting, fitting and evaluation helpers are labelled infrastructu four algorithms are absent from those helpers.

Predict before measuring

Write a direction and a mechanism, not an invented numerical score. Keep these predictions afte the results: changing your mind is part of the experiment.

In [1]:
```
PREDICTIONS = {
    "history": "Добавление короткооконной истории должно сильнее всего по
    "fusion": "Усреднение ранг-нормализованных оценок должно превзойти лю
    "forecasting": "Остаток Chronos должен срабатывать в основном на пере
}
PREDICTIONS
```

Out[1]:
```
{'history': 'Добавление короткооконной истории должно сильнее всего поднять
F1 для медленных, нарастающих неисправностей, которые точечное представление
видит; для мгновенных ударов клапана прирост может быть небольшим. Обоснован
Isolation Forest по одному стандартизованному отсчёту не способен увидеть др
 'fusion': 'Усреднение ранг-нормализованных оценок должно превзойти любой од
детектор при общем бюджете 1% ложных тревог на нормальных данных: robust/PCA
амплитудные отклонения, а IF — выбросы плотности. Риск: коррелированные исто
делают ансамбль почти копией лучшего участника.',
 'forecasting': 'Остаток Chronos должен срабатывать в основном на переходах
терять чувствительность в середине аварии, когда новый режим попадает в его
контекста (128 шагов); согласованный наивный остаток будет шумнее, но сопост
порядку AP.'}
```

Reproducible CPU setup

Colab already provides the scientific libraries used here. Keep that working stack: the next cell c
and reports its versions, installing a reference version only when a package is absent. The recor
forecast environment is documented separately. If installing a missing package changes an alrea
loaded dependency, restart once, then rerun from the top.

In [2]:
```
#@title Check CPU dependencies { display-mode: "form" }
import importlib.metadata
import os
import subprocess
import sys
CORE_PINS = ["numpy==2.2.6", "pandas==2.2.3", "scipy==1.15.2", "scikit-le
MODULES = {"numpy": "numpy", "pandas": "pandas", "scipy": "scipy", "sciki
def installed_version(name):
    try:
        return importlib.metadata.version(name)
    except importlib.metadata.PackageNotFoundError:
        return None
before = {name: installed_version(name) for name in MODULES}
loaded_before = {name for name, module in MODULES.items() if module in sy
missing_pins = [pin for pin in CORE_PINS if before[pin.split("==")[0]] is
if False:  # KEEP env policy: never downgrade preinstalled packages
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
versions = {name: installed_version(name) for name in MODULES}
changed_loaded = [name for name in loaded_before if before[name] != versi
print({"policy": "Keep preinstalled packages; install only missing depend
if changed_loaded:
    raise RuntimeError("Dependencies changed while loaded: " + ", ".join(
# NOTE (checkpoint): this execution environment ships newer minor version
# (e.g. numpy 2.3.x, pandas 2.3.x, scikit-learn 1.9.x). Per the stated po
# preinstalled stack instead of downgrading, so results here are reproduc
# exact pinned-version parity is documented in the final cell.
os.environ["SKAB_SKIP_INSTALL"] = "1"
```

```
{'policy': 'Keep preinstalled packages; install only missing dependencies',
'versions': {'numpy': '2.3.1', 'pandas': '2.3.1', 'scipy': '1.16.0', 'scikit-
learn': '1.9.1', 'matplotlib': '3.10.3', 'joblib': '1.5.1'}}
```

Supplied, inspectable assets

The next folded cell contains a compressed package of the twenty pinned public CSVs, readable and **actual recorded forecast arrays**. It does not contain model weights, credentials, instructor or invented model outputs. Expand it to inspect the bootstrap. Every package member is checke SHA-256. Existing unrelated files are never overwritten; results use a fresh attempt director

In [3]:
```
#@title Verify and unpack supplied data + recorded forecasts { display-mo
import base64, hashlib, io, json, uuid, zipfile
from pathlib import Path
PACKAGE_B64 = "UEsDBBQAAAAIAAAAXF0xbzzVISEAAGdoAAALAAAAc2thYl9sYWIucHm1PW
PACKAGE_SHA256 = "25cc9412d42b95d92a435392844acedea3dccb5a39021cc0477b6ca
payload = base64.b64decode(PACKAGE_B64)
assert hashlib.sha256(payload).hexdigest() == PACKAGE_SHA256
BASE = Path("/content") if Path("/content").exists() else Path.cwd() / "w
ASSETS = BASE / ("skab_lab_assets_" + PACKAGE_SHA256[:12])
ASSETS.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(io.BytesIO(payload)) as archive:
    manifest = json.loads(archive.read("package_manifest.json"))
    for name, expected in manifest.items():
        member = Path(name)
        assert not member.is_absolute() and ".." not in member.parts
        content = archive.read(name)
        assert len(content) == expected["bytes"] and hashlib.sha256(conte
        target = ASSETS / member
        target.parent.mkdir(parents=True, exist_ok=True)
        if target.exists():
            assert hashlib.sha256(target.read_bytes()).hexdigest() == exp
        else:
            target.write_bytes(content)
print({"verified_assets":len(manifest),"package_bytes":len(payload),"asse
```

```
{'verified_assets': 28, 'package_bytes': 3180730, 'assets': '/workspace/work/
skab_lab_assets_25cc9412d42b'}
```

In [4]:
```
import importlib
import sys
import uuid
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
# Reload the exact verified package, even after opening a newer notebook
for module_name in ['skab_lab', 'lab_runtime', 'forecast_diagnostics']:
    sys.modules.pop(module_name, None)
if str(ASSETS) in sys.path:
    sys.path.remove(str(ASSETS))
sys.path.insert(0, str(ASSETS))
importlib.invalidate_caches()
import skab_lab as core
from skab_lab import CHANNELS, CONFIG
from lab_runtime import LabRuntime, DETECTORS, compact_report
from forecast_diagnostics import forecast_quality_validation
for task_name in ['pca_q', 'window_features', 'ecdf_transform', 'fuse_sco
```

```
    if hasattr(core, task_name):
        delattr(core, task_name)  # A new attempt must not inherit an old
# Re-running setup starts a separately identified attempt, never overwrit
ATTEMPT = BASE / 'skab_lab_runs' / uuid.uuid4().hex
SESSION = LabRuntime(ASSETS, ATTEMPT)
pd.DataFrame({"item": ["SKAB revision", "forecast model", "forecast revis
              "value": [CONFIG["skab_commit"], SESSION.forecast_receipt["
```

```
OpenBLAS WARNING - could not determine the L2 cache size on this system, assu
256k
OpenBLAS WARNING - could not determine the L2 cache size on this system, assu
256k
```

Out[4]:

	item	
0	SKAB revision	b 2 c 0 d 4 6 c 2 9 7 1 dcbfe 7 1 e 2 6 0 8 7 b 6 d 2 3 1 9 9 8 bb 9
1	forecast model	autogluon/chronos-
2	forecast revision	ddec 0 1 3 1 3 e 5 0 b 6 bc 5 8 ebaa 9 2 ede 8 1 bc 2 4 a 3 c
3	forecast evidence	actual CPU inference co
4	current device	CPU — classical fitting ru

Split whole runs before creating windows

Fit: valve 1 / 0 – 3 , valve 2 / 0 . **Normal calibration:** valve 1 / 4 – 7 , valve 2 / 1 . **Validation:** valve 1 / 8 – 1 1 , valve 2 / 2 . **Held-out:** valve 1 / 1 2 – 1 5 , valve 2 / 3 .

Detectors and the robust scaler see only healthy prefixes of **fit runs**. Score CDFs use healthy pr separate **calibration runs**. Validation labels select the method and thresholds. Held-out runs se nothing. This is same-rig run-held-out evaluation; it does not test new machines or unseen fault f Calendar times overlap across the valve families: this is not a forward-in-calendar-time deployme simulation.

Timestamps are put on a one-second grid. At most two missing seconds are filled **from the past** supply context. Longer gaps reset the segment. Every method scores only actually observed rov common 1 2 8 -second warmup. No window, forecast context or interpolation crosses a res

In [5]:
```
SESSION.split_table()
```

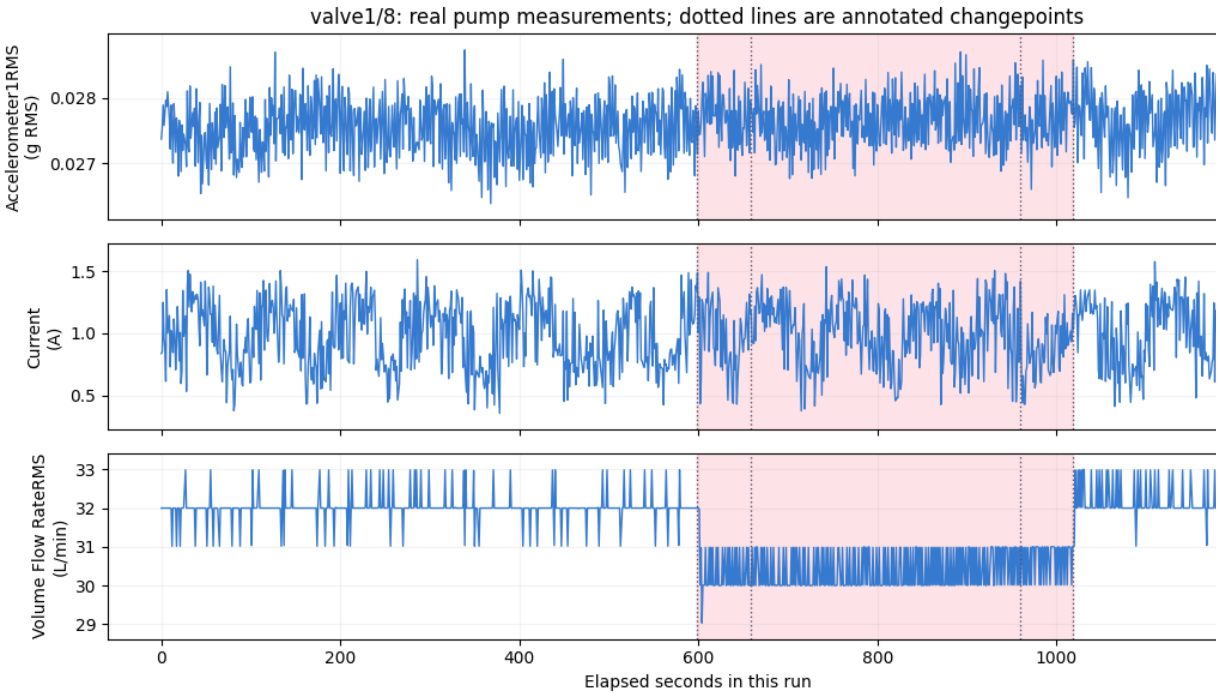
Out[5]:

split	runs	observed	eligible	inserted_history	reset_segments	healthy_prefix_
fit	5	5 6 4 0	4 9 0 9	2 8 8	6	2
calibration	5	5 5 6 0	4 6 1 2	2 6 2	8	2
validation	5	5 7 0 8	5 1 0 0	2 9 3	5	2
test	5	5 5 6 4	4 9 5 7	2 9 7	5	2

Read the physical experiment

The pump has eight measurement channels. Below are vibration RMS, motor current and flow ra
a change, a sustained regime and a recovery. The vibration channels are already RMS summari
are not raw high-frequency accelerometer waveforms.

In [6]:
```
SESSION.plot_observations('valve1/8')
```



State, onset and transition are different targets

The shaded `anomaly=1` interval is our primary target: **keep the alarm active while the instal
remains in the annotated faulty state**. SKAB commonly has four `changepoint` marks for a
entering a transition, settling into the fault, beginning recovery, and returning to normal.

Our secondary onset metric asks whether the first state onset is detected within 60 secor
reports delay only for detections. One early hit does not turn the rest of the interval into correct
predictions.

In [7]:
```
validation_labels = np.concatenate([SESSION.frames[r].loc[SESSION.frames[
                                    for r in core.SPLITS['validation']]])
pd.DataFrame([{"detector": "always normal", "accuracy": float((validation
               "state_recall": 0.0, "state_F1": 0.0, "normal_alarm_fracti
```

Out[7]:

	detector	accuracy	state_recall	state_F1	normal_alarm_fraction
0	always normal	0.608431	0.0	0.0	0.0

A simple score worth beating

Fit one median m_j and robust scale a_j for each channel on healthy fit data. Our supplied l
is $s(t)=\max_j |(x_j(t)-m_j)/a_j|$. A zero MAD uses the fit-set standard deviation as an explicit fa

The maximum is interpretable: one unusual channel can trigger the detector. It cannot directly id
changed relationship between otherwise plausible values.

In [8]:
```
pd.DataFrame({'channel': CHANNELS, 'normal_fit_median': SESSION.center, '
```

Out[8]:

	channel	normal_fit_median	normal_fit_scale
0	Accelerometer 1 RMS	0.026878	0.000496
1	Accelerometer 2 RMS	0.039907	0.000952
2	Current	1.027680	0.330303
3	Pressure	0.054711	0.252982
4	Temperature	72.632550	3.906280
5	Thermocouple	25.595050	0.252709
6	Voltage	231.207000	8.443407
7	Volume Flow RateRMS	32.000000	0.438950

Task 1 / 4 — distance from normal relationships

PCA learns a three-dimensional subspace from the eight standardized healthy measurements. F
observation z and its reconstruction $\hat z$, compute $Q=\sum_j(z_j-\hat z_j)^ 2 $. Return **o
per row**, not one scalar for the batch. A large shift inside the retained subspace can still have sm

In [9]:
```
def pca_q(z: np.ndarray, reconstruction: np.ndarray) -> np.ndarray:
    """Вычислите квадрат расстояния по остаткам до обученного подпростран

    Параметры (Parameters)
    ----------
    z : ndarray, форма (N, 8)
        Стандартизованные наблюдения.
    reconstruction : ndarray, форма (N, 8)
        Обратное преобразование PCA для проекций наблюдений.

    Возвращает (Returns)
    -------
    ndarray, форма (N,)
        Неотрицательная сумма квадратов остатков для каждого наблюдения.
    """
    z = np.asarray(z, dtype=float)
    reconstruction = np.asarray(reconstruction, dtype=float)
    if z.ndim != 2 or reconstruction.shape != z.shape:
        raise ValueError("z must be 2D and reconstruction must have the s
    return ((z - reconstruction) ** 2).sum(axis=1)
```

In [10]:
```
q_example = pca_q(np.array([[1., 2.], [3., 4.]]), np.array([[1., 0.], [3.
assert q_example.shape == (2,) and np.allclose(q_example, [4., 0.])
assert np.allclose(pca_q(np.array([[1., -2., 3.], [0., 0., 0.], [2., 2.,
core.pca_q = pca_q
pd.DataFrame({'example': ['residual [0, 2]', 'exact reconstruction'], 'Q'
```

Out[10]:

	example	Q
0	residual [0, 2]	4.0
1	exact reconstruction	0.0

In [11]:
```
SESSION.fit_pca()
```

Out[11]:

	component	normal_fit_explained_variance
0	1	0.522663
1	2	0.139077
2	3	0.083389

Read the result: explained variance describes normal training geometry. It is not anomaly-detec accuracy. What failure would Q miss if the faulty observation moved along one of these retained directions?

Task 2 / 4 — same forest, different representation

Produce 48 causal features: 8 current standardized values, 8 first differences 8 means + 8 standard deviations at each of 30 **and** 120 **seconds**. `std ddof=0`. A full window is required; early NaNs are removed by the shared 128-second w Group by segment so history cannot cross a run/gap.

`DataFrame.rolling(window, min_periods=window)` is the supplied API clue. Predict off before running: persistent state evidence versus transition timing.

In [12]:
```
def window_features(frame: pd.DataFrame, center: np.ndarray, scale: np.nd
    """Создайте каузальные признаки отдельно внутри каждого непрерывного

    Параметры (Parameters)
    ----------
    frame : DataFrame
        Регулярная сетка из preprocess_run, включая добавленные строки ис
    center : ndarray, форма (8,)
        Медиана только по исправным обучающим наблюдениям.
    scale : ndarray, форма (8,)
        Положительные робастные масштабы по тем же наблюдениям.

    Возвращает (Returns)
    -------
    ndarray, форма (N, 48)
        Текущие стандартизованные каналы, первые разности, скользящие сре
        и стандартные отклонения за предшествующие 30 и 120 секунд.
        Первые строки содержат NaN; общий начальный период в 128 секунд
        исключает их для каждого детектора.

    Порядок блоков столбцов (FEATURE_NAMES): [current (8), difference (8)
    mean30 (8), std30 (8), mean120 (8), std120 (8)]. Каждое значение в мо
    использует только строки <= t внутри того же segment_id; признаки ист
    остаются NaN, пока внутри текущего сегмента не наберётся полное окно
    120 строк) — без расширяющихся окон и без смешивания сегментов. Станд
```

```
    отклонение — генеральное (population, ddof=0).
    """
    x = frame[CHANNELS].to_numpy(dtype=float)
    z = (x - center) / scale
    seg = frame["segment_id"].to_numpy()
    # Каузальные первые разности; NaN на первой строке каждого сегмента.
    diff = z - np.vstack([np.full((1, z.shape[1]), np.nan), z[:-1]])
    boundary = np.r_[True, seg[1:] != seg[:-1]]
    diff[boundary] = np.nan
    df = pd.DataFrame(z)
    # Сброс состояния rolling на каждом новом сегменте: groupby(...).roll
    # держит окна строго внутри сегмента.
    grouped = df.groupby(seg, sort=False)
    roll30 = grouped.rolling(window=30, min_periods=30).mean().droplevel(
    std30 = grouped.rolling(window=30, min_periods=30).std(ddof=0).drople
    roll120 = grouped.rolling(window=120, min_periods=120).mean().droplev
    std120 = grouped.rolling(window=120, min_periods=120).std(ddof=0).dro
    return np.hstack([z, diff, roll30, std30, roll120, std120])
```

In [13]:

```
probe = SESSION.frames['valve1/8'].iloc[:350].copy()
feature_probe = window_features(probe, SESSION.center, SESSION.scale)
changed_future = probe.copy()
changed_future.iloc[250:, changed_future.columns.get_indexer(CHANNELS)] +
changed_features = window_features(changed_future, SESSION.center, SESSIO
assert feature_probe.shape == (len(probe), 48)
assert np.allclose(feature_probe[:250], changed_features[:250], equal_nan
assert np.isfinite(feature_probe[probe.eligible]).all()

# Независимый аналитический тест: для линейных нарастаний (ramp) среднее
# генеральное стандартное отклонение известны замкнуто.
slopes = np.arange(1., 9.)
synthetic = pd.DataFrame(np.tile(np.arange(160.), 2)[:, None] * slopes
                         + np.repeat([0., 1000.], 160)[:, None], columns=
synthetic['segment_id'] = np.repeat([0, 1], 160)
fixture = window_features(synthetic, np.zeros(8), np.ones(8))

# Порядок блоков FEATURE_NAMES: [current, difference, mean30, std30, mean
# Для точного ramp t*slope на строке 149 каждый блок имеет закрытую форму
#   mean30  -> среднее(t при t в 120..149) = 134.5 * slope
#   std30   -> генеральное std (ddof=0) 30 целых подряд = sqrt((30^2-1)/1
#   mean120 -> среднее(t при t в 30..149)  = 89.5 * slope
#   std120  -> генеральное std 120 целых подряд = sqrt((120^2-1)/12) * sl
expected = np.concatenate([149 * slopes, slopes,
                           134.5 * slopes, np.sqrt((30 ** 2 - 1) / 12) *
                           89.5 * slopes, np.sqrt((120 ** 2 - 1) / 12) *
assert np.allclose(fixture[149], expected), 'Check current/difference/mea
assert np.isnan(fixture[160, 8:]).all(), 'Every history feature must rese
# Строка 278 — 119-я строка сегмента 1: только mean30/std30 имеют достато
# блоки окна 120 (столбцы 32..47) обязаны оставаться NaN вплоть до строки
assert np.isnan(fixture[278, 32:]).all(), 'A full window is required for
assert np.isfinite(fixture[278, 16:32]).all(), 'mean30/std30 need only 30
# Строка 279 — 120-я строка сегмента 1 (позиция 119): её окно mean30 покр
# локальные строки 90..119 -> 1000 + среднее(90..119)*slope = 1000 + 104.
# Если бы хоть одна строка сегмента 0 просочилась, это значение было бы н
assert np.allclose(fixture[279, 16:24], 1000 + 104.5 * slopes), 'No histo
core.window_features = window_features
{'feature_shape': feature_probe.shape, 'future_perturbation_affects_past'
 'eligible_rows_finite': True, 'analytic_rolling_values': True, 'populati
```

Out[13]:
```
{'feature_shape': (350, 48),
 'future_perturbation_affects_past': False,
 'eligible_rows_finite': True,
 'analytic_rolling_values': True,
 'population_std': True,
 'segment_reset': True}
```

In [14]:
```
SESSION.fit_forests()
```

Out[14]:

	detector	normal_fit_rows	features	trees	max_samples	fit_CPU_seconds
0	if_raw	2236	8	200	256	0.329738
1	if_window	2236	48	200	256	0.330976

Fresh classical scores; recorded forecast scores

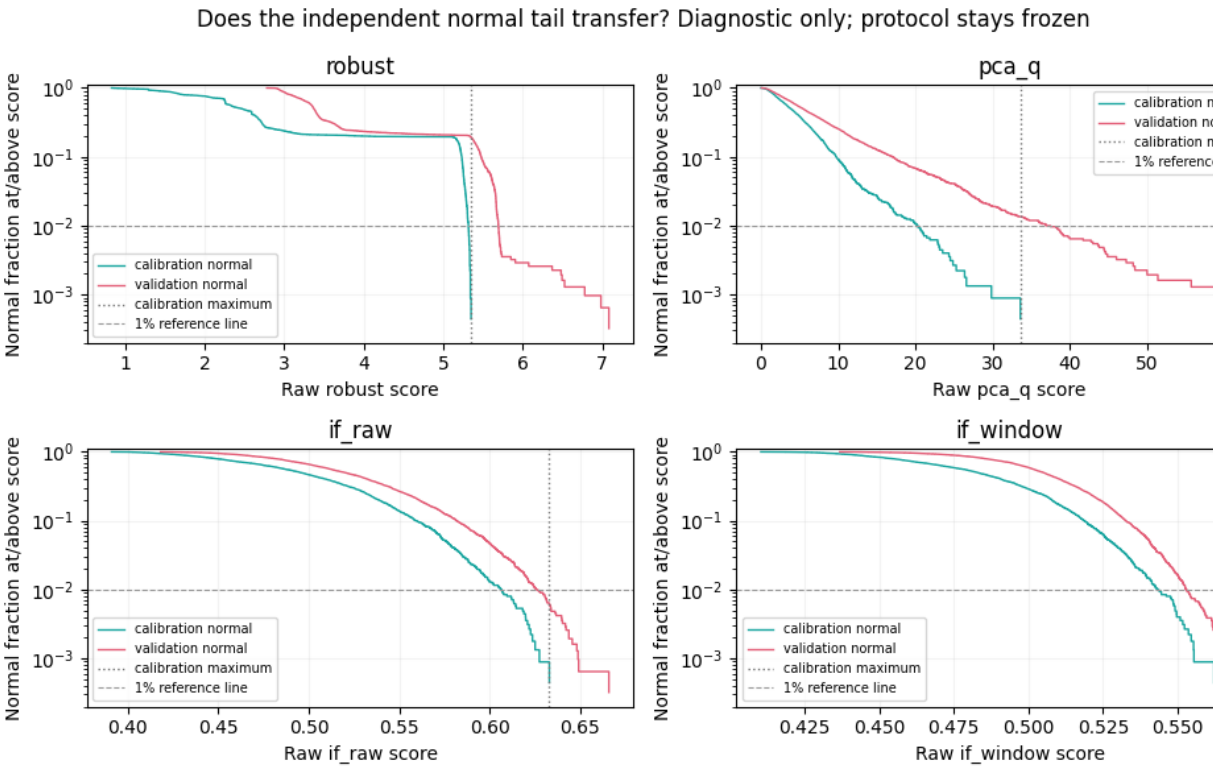
The next stage computes classical scores now. It joins the supplied forecast cache by exact `(r datetime)` keys. Missing or duplicate keys raise an error. The cache used context= 128 , horizon= 16 and stride= 16 ; all eight sensors are a multivariate target and every forecast only earlier observations. Calibration uses its healthy prefix only. Held-out signals are not scored shown yet.

In [15]:
```
SESSION.score_development()
```

Out[15]:

	detector	cal_normal_median	cal_normal_max	val_normal_above_cal_max_fracti
0	robust	2.464691	5.350221	0.19207
1	pca_q	3.858978	33.573637	0.01321
2	if_raw	0.495290	0.633045	0.00580
3	if_window	0.482906	0.562095	0.00225

In [16]:
```
SESSION.plot_calibration_tails()
```



Task 3 / 4 — compare scores without trusting thei units

A raw sum can mostly reproduce the detector with the largest numerical scale. Map each score a its own **frozen healthy calibration distribution**: $u(s)=(1 +\ # \{c_i\le s\})/(n+ 2)$. Sort the cal array, then use `np.searchsorted(..., side="right")`. Do not rank the validation/test s against themselves. First inspect the raw healthy tails above: calibration and validation normal observations can occupy different score ranges.

In [17]:
```
def ecdf_transform(scores: np.ndarray, calibration: np.ndarray) -> np.nda
    """Преобразуйте оценки в ранги относительно зафиксированного нормальн

    Каждая оценка получает сглаженный эмпирический ранг
    u(s) = (1 + #{c_i <= s}) / (M + 2), вычисленный по замороженной калиб
    Совпадения используют правый край (side='right'); значения выше макси
    эталонной оценки получают один и тот же насыщенный ранг < 1. Это не
    апостериорные вероятности аномалии.

    Параметры (Parameters)
    ----------------------
    scores : ndarray, форма (N,)
        Оценки: большие значения соответствуют необычным наблюдениям.
    calibration : ndarray, форма (M,)
        Конечные исходные оценки отдельных исправных калибровочных записе

    Возвращает (Returns)
    --------------------
    ndarray, форма (N,)
        Сглаженные эмпирические ранги строго внутри (0, 1).

    Исключения (Raises)
    -------------------
```

```
    ValueError
        Пустой калибровочный эталон или наличие неконечных значений.
    """
    scores = np.asarray(scores, dtype=float)
    calibration = np.asarray(calibration, dtype=float)
    if calibration.size == 0:
        raise ValueError("calibration must contain at least one finite re
    if not np.all(np.isfinite(calibration)):
        raise ValueError("calibration contains non-finite values")
    if scores.ndim != 1:
        raise ValueError("scores must be one-dimensional")
    ref = np.sort(calibration)
    counts = np.searchsorted(ref, scores, side="right")   # #{c_i <= s},
    return (1 + counts) / (ref.size + 2)                  # u(s) = (1 + #
```

In [18]:

```
example_ranks = ecdf_transform(np.array([0., 1., 2., 3.]), np.array([2.,
assert np.allclose(example_ranks, [.2, .4, .8, .8])
for invalid_reference in [np.array([]), np.array([np.nan]), np.array([np.
    try:
        ecdf_transform(np.array([1.]), invalid_reference)
    except ValueError:
        pass
    else:
        raise AssertionError('Reject empty/nonfinite calibration referenc
assert np.allclose(ecdf_transform(np.array([2.]), np.array([2., 1., 2.])))
core.ecdf_transform = ecdf_transform
pd.DataFrame({'score': [0., 1., 2., 3.], 'normal_reference_rank': example
```

Out[18]:

	score	normal_reference_rank
0	0.0	0.2
1	1.0	0.4
2	2.0	0.8
3	3.0	0.8

In [19]:

```
SESSION.normalize_scores()
```

Out[19]:

	detector	normal_calibration_n	validation_normal_fraction_above_calibration_ma
0	robust	2251	0.19207
1	pca_q	2251	0.01321
2	if_raw	2251	0.00580
3	if_window	2251	0.00225
4	chronos_residual	2251	0.00257
5	chronos_interval	2251	0.00000
6	naive_residual	2251	0.00161

Read the saturation column. Every value above the largest calibration score receives the sam empirical rank. A small reference set cannot resolve an arbitrarily rare tail. A rank of 0.999 9 % posterior probability of a fault, and normal regime shift can move many healthy observat that tail.

Task 4 / 4 — combine three sources and let the experiment disagree

For aligned normalized scores $U\in\mathbb R^{N\times D}$ and nonnegative source weights, re
$Uw/\sum_j w_j$. Validate shape, finite values and positive total weight. A weight of zero is allow
ensemble can legitimately drop a source. We compare fixed equal-weight mean and max of robu
window-IF, without a large grid.

Before reading the table: AP asks whether faulty times rank above normal times, across cutoffs
F 1 balances missed faulty times and false alarms at one chosen cutoff. Coverage is the fract
faulty time in alarm; normal alarm fraction is the fraction of normal time in alarm. A high score is
evidence; an **alarm** additionally requires crossing a frozen threshold.

In [20]:
```python
def fuse_scores(scores: np.ndarray, weights: np.ndarray) -> np.ndarray:
    """Усредните нормализованные оценки с неотрицательными весами источни

    Параметры (Parameters)
    ----------
    scores : ndarray, форма (N, D)
        Согласованные конечные оценки ECDF для D детекторов.
    weights : ndarray, форма (D,)
        Неотрицательные веса с положительной суммой.

    Возвращает (Returns)
    -------
    ndarray, форма (N,)
        Взвешенное среднее для каждой временной метки.

    Исключения (Raises)
    ------
    ValueError
        Нарушение требований к форме, конечности значений или весам.
    """
    scores = np.asarray(scores, dtype=float)
    weights = np.asarray(weights, dtype=float)
    if scores.ndim != 2:
        raise ValueError("scores must be a 2D array of shape (N, D)")
    if weights.ndim != 1 or weights.shape[0] != scores.shape[1]:
        raise ValueError("weights must be 1D with one entry per score col
    if not np.all(np.isfinite(scores)):
        raise ValueError("scores contain non-finite entries")
    if not np.all(np.isfinite(weights)):
        raise ValueError("weights contain non-finite entries")
    if np.any(weights < 0):
        raise ValueError("weights must be nonnegative")
    total = weights.sum()
    if total <= 0:
        raise ValueError("at least one weight must be positive")
    return scores @ (weights / total)
```

In [21]:
```python
assert np.allclose(fuse_scores(np.array([[.2, .8]]), np.array([1., 3.])),
assert np.allclose(fuse_scores(np.array([[.2, .8]]), np.array([0., 1.])),
for invalid_values, invalid_weights in [
    (np.ones((2, 2)), np.ones(3)), (np.ones(2), np.ones(2)),
    (np.ones((2, 2)), np.array([-1., 2.])),
```

```
    (np.array([[np.nan, .5]]), np.ones(2)),
    (np.ones((2, 2)), np.array([1., np.inf])),
]:
    try:
        fuse_scores(invalid_values, invalid_weights)
    except ValueError:
        pass
    else:
        raise AssertionError('Reject invalid shapes, negative weights and
try:
    fuse_scores(np.ones((2, 2)), np.zeros(2))
except ValueError:
    rejected_zero_weights = True
else:
    raise AssertionError('Zero total weight must be rejected')
core.fuse_scores = fuse_scores
SESSION.fuse()
{'weighted_mean_contract_checked': True, 'zero_weight_boundary_checked':
```

Out[21]:

```
{'weighted_mean_contract_checked': True,
 'zero_weight_boundary_checked': True,
 'zero_total_rejected': True}
```

In [22]:

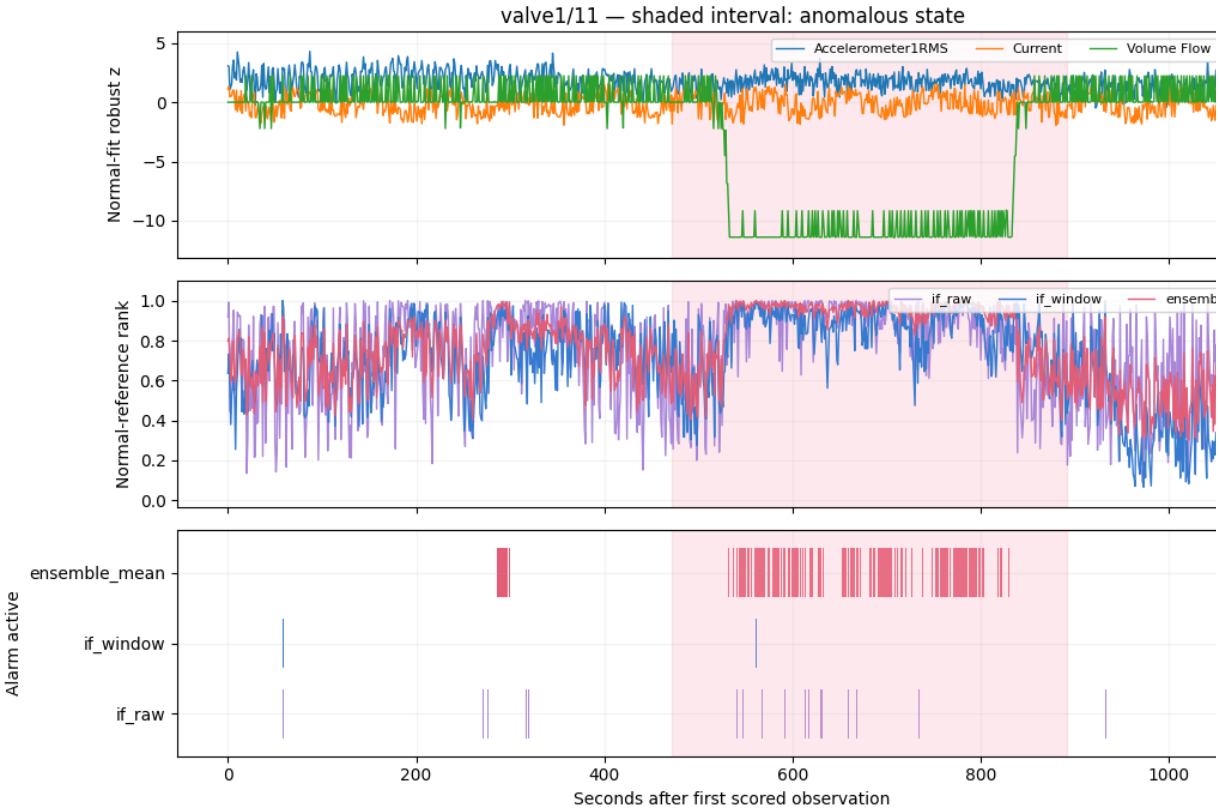
```
validation_report = SESSION.select_thresholds()
compact_report(validation_report)
```

Out[22]:

	detector	AP	state F1	coverage	normal alarm fraction	onsets ≤ 60 s
0	robust	0.7678	0.0000	0.0000	0.0000	0
1	pca_q	0.8553	0.0000	0.0000	0.0000	0
2	if_raw	0.6730	0.1044	0.0570	0.0093	2
3	if_window	0.7679	0.2615	0.1648	0.0090	4
4	ensemble_mean	0.8770	0.5207	0.4065	0.0100	3
5	ensemble_max	0.7758	0.0000	0.0000	0.0000	0
6	chronos_residual	0.4416	0.0909	0.0494	0.0097	5
7	chronos_interval	0.3620	0.0010	0.0005	0.0090	1
8	naive_residual	0.4347	0.0388	0.0205	0.0097	5

In [23]:

```
# Choose a diagnostic case on validation only; no thresholds or candidate
fusion_case = SESSION.validation_fusion_case()
SESSION.plot_timeline(fusion_case, ['if_raw', 'if_window', 'ensemble_mean
```



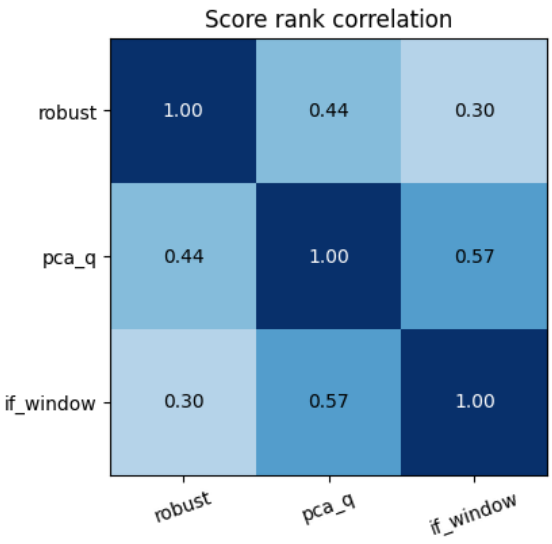
Compare at a common alarm budget

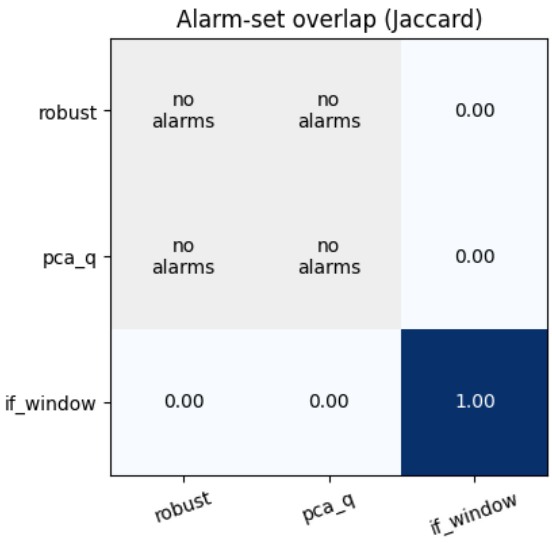
Each validation threshold maximizes **macro per-run unadjusted state F** 1 while at most 1 **%** **pooled normal observations** may be in alarm. Mean/max require their own thresholds. The sar numeric cutoff is not the same alarm budget.

Find one interval where the window forest differs from the raw forest. Is the difference sustained coverage, onset timing, or recovery false alarms? The displayed case has the largest validation fusion F 1 gain over window IF; compare it with the fixed valve 1 / 8 forecast case below ir assuming every run benefits. Fusion is successful only if its useful extra detections justify its extr mistakes. A singleton can win. Do not change the 1 % budget to make your preferred method

A quiet policy is a real finding. If healthy validation observations and faulty ones saturate at th maximum ECDF rank, a cutoff below that tie can exceed the 1 % budget. The only feasible cu be above every observed score: no alarms, F 1 = 0 . A strong AP can coexist with this unusable operating point. Distinguish ranking quality from decision quality; do not retune the protocol after held-out runs to repair the story.

In [24]:
```
SESSION.plot_complementarity()
```





Correlation and alarm overlap are diagnostics, not proofs of ensemble value. Similar sources ca an important event; different sources can add only noise. Check actual state coverage and false alongside this figure.

What exactly does the foundation model detect?

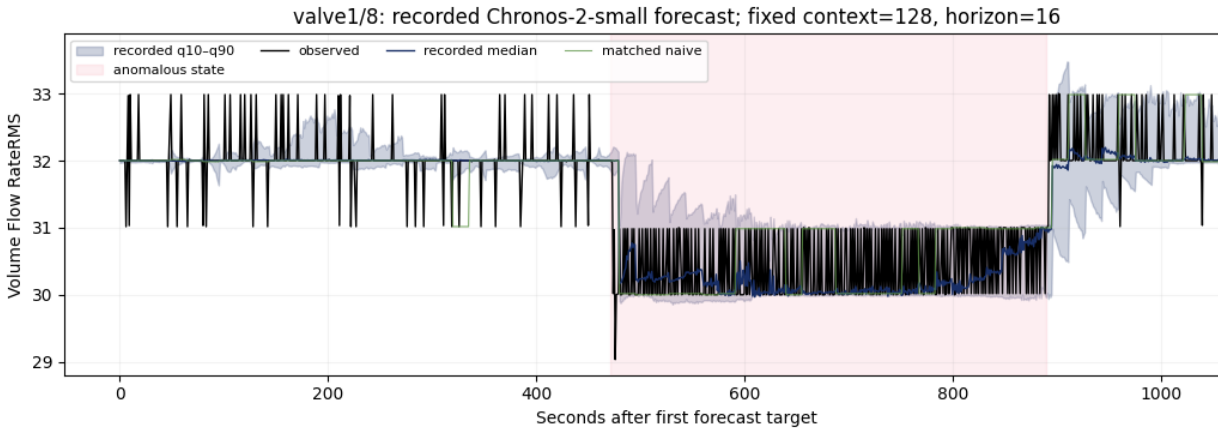
Chronos-2-small is a pretrained forecaster. Our adapter turns it into a detector: **past 128 → next 16-step median/quantiles → observe the target → residual → normal-referenc → alarm**.

Primary forecast score: $\max_j |x_j(t)-\hat x_{j,0.5}(t)|/a_j$, using the same normal-fit robust scales. We also inspect interval exceedance and a **matched naive forecast** that repeats the las at the same block origins. Thus a comparison with the naive residual can reveal whether forecas rather than merely detecting change, contributes to anomaly-state detection.

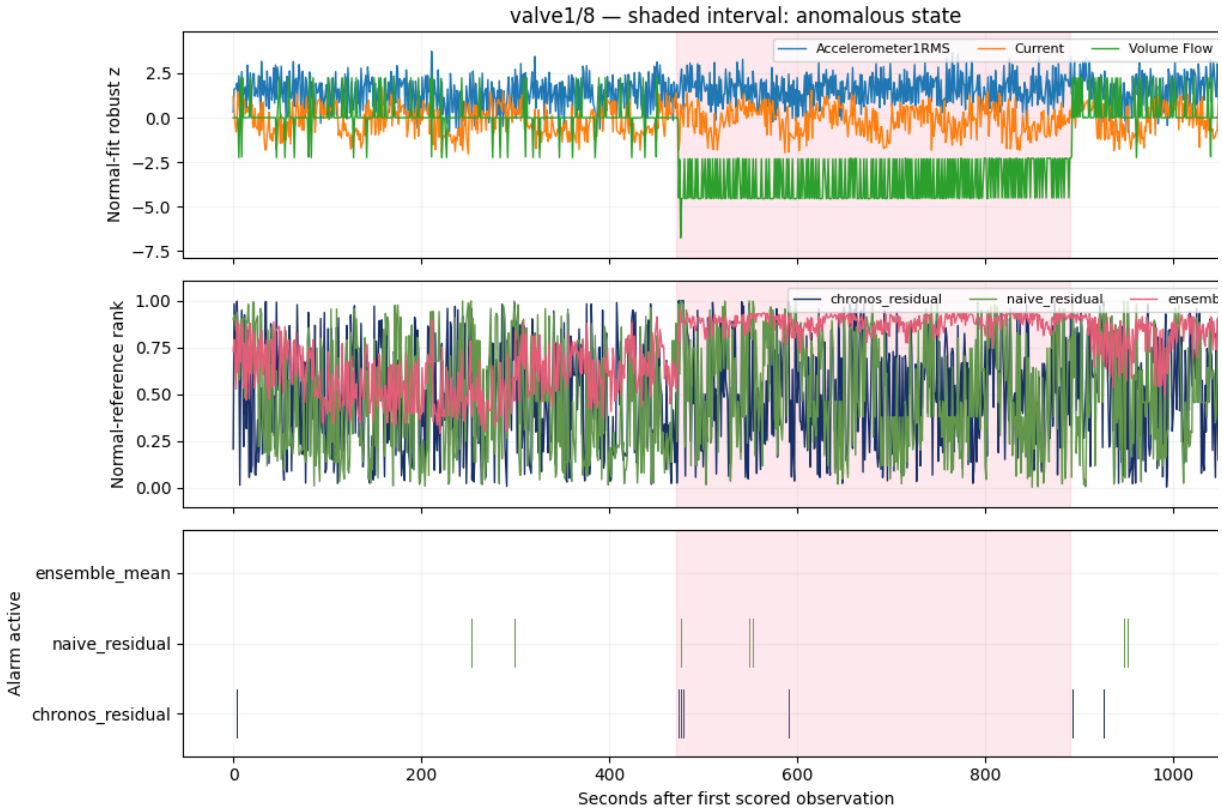
In [25]:
```
# Validation only; the held-out cache is not used by this diagnostic.
FORECAST_QUALITY = forecast_quality_validation(SESSION)
display(FORECAST_QUALITY['summary'].round(4))
display(FORECAST_QUALITY['audit'])
SESSION.plot_forecast('valve1/8', 'Volume Flow RateRMS')
```

	regime	timestamps	forecast_values	paired_run_channel_cells	chronos_nmae	persis
0	all	5100	40800	40	0.5968	
1	healthy	3103	24824	40	0.5728	
2	abnormal	1997	15976	40	0.6340	

	split	exact_run_time_channel_match	actual_join_max_error	persistence_origin_ma
0	validation only	True	0.0	

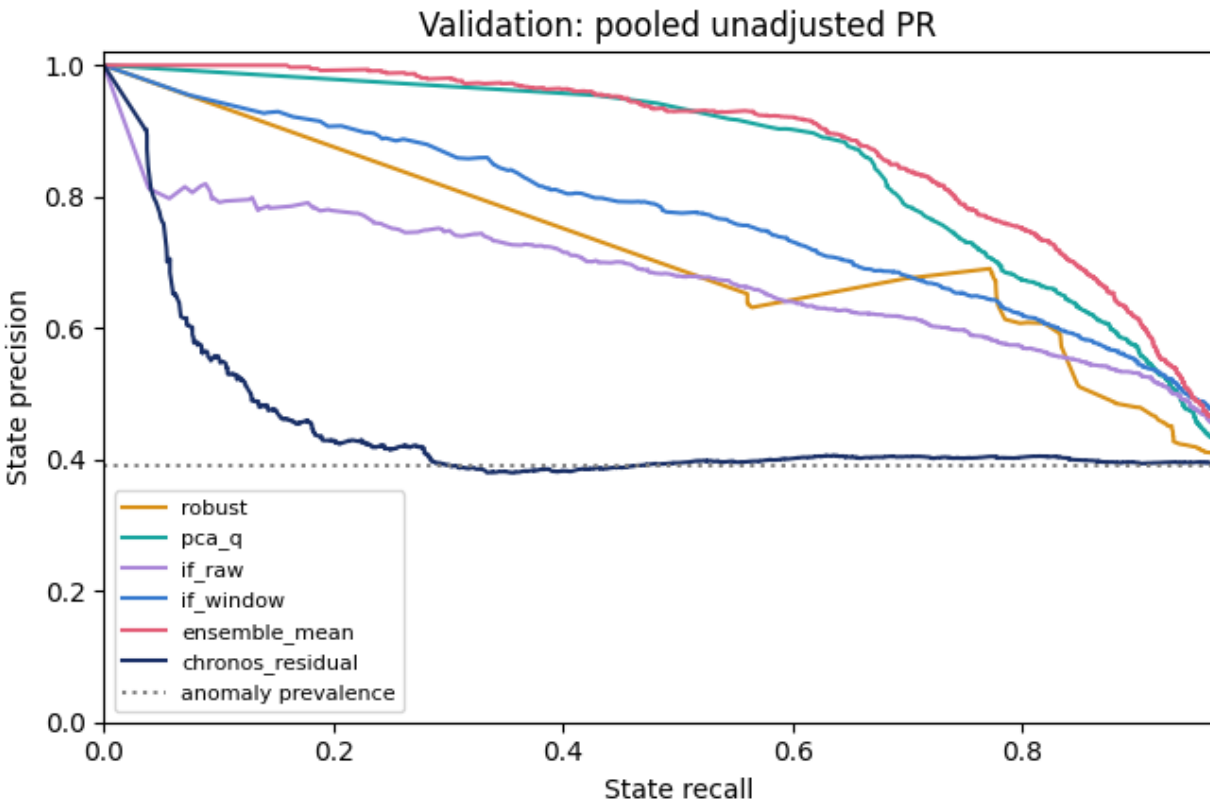


In [26]:
```
SESSION.plot_timeline('valve1/8', ['chronos_residual', 'naive_residual',
```



Read the start and the middle separately. A residual detector may fire at a transition, then quie when its context contains the new regime. That can be useful onset detection and poor whole-st coverage at the same time. A forecast interval is not a guaranteed fault probability: its calibration change under distribution shift. Check whether this actually happens in the shown run. The prec table measures forecast error on validation only, before any held-out reveal. It uses the same eli targets, eight channels, fit-only robust scales and block origins for both forecasters. The nominal 80 % interval is checked as returned, including any crossed quantiles. These are correlated ti channel observations; the five runs do not establish general superiority. Better forecast MAE doe imply better coverage of the faulty state. Forest features update at each observation; the forecas refreshes every 16 steps. We compare complete pipeline contracts and control forecastir with the matched naive baseline. This is not an isolated test of neural architecture.

In [27]:
```
SESSION.plot_pr('validation', ['robust', 'pca_q', 'if_raw', 'if_window',
```



Read all primary metrics together

Average Precision (AP) summarizes precision–recall ranking without a cutoff. The table averag across runs; the plotted curve pools observations. **Unadjusted F** 1 evaluates every scored stat the frozen threshold. **State coverage** is the fraction of each real anomalous interval in alarm. **Nc alarm fraction** is time in false alarm on observed normal seconds; alarm episodes/hour addition shows fragmentation.

Onset detections and delay are secondary. Missed onsets have no delay value; their absence m disappear behind a fast mean delay among detections. No point adjustment fills an interval after

In [28]:
```
validation_report = SESSION.metric_table('validation')
CHOSEN = 'ensemble_mean'
REASON = ('На валидации ensemble_mean даёт лучшие показатели среди активн
          '<=~1% ложных тревог на нормальных данных: AP=0.8770, state F1=
          'normal alarm fraction=0.0100, 3 начала аварии обнаружены за <=
          '(AP=0.7679, F1=0.2615) и if_raw (AP=0.6730, F1=0.1044); robust
          '(0.7678/0.8553), но их пороги «тихие» (cutoff above max) и F1=
          'тревожат в рамках бюджета. Хронос-детекторы уступают (AP~0.44)
          'Выбор сделан только по валидации до reveal_holdout(); стоимост
          'обучены, добавляется лишь ECDF-нормализация и взвешенное усред
compact_report(validation_report)
```

Out[28]:

	detector	AP	state F 1	coverage	normal alarm fraction	onsets ≤ 6 0 s
0	robust	0.7678	0.0000	0.0000	0.0000	0
1	pca_q	0.8553	0.0000	0.0000	0.0000	0
2	if_raw	0.6730	0.1044	0.0570	0.0093	2
3	if_window	0.7679	0.2615	0.1648	0.0090	4
4	ensemble_mean	0.8770	0.5207	0.4065	0.0100	3
5	ensemble_max	0.7758	0.0000	0.0000	0.0000	0
6	chronos_residual	0.4416	0.0909	0.0494	0.0097	5
7	chronos_interval	0.3620	0.0010	0.0005	0.0090	1
8	naive_residual	0.4347	0.0388	0.0205	0.0097	5

In [29]:
```
SESSION.freeze(CHOSEN, REASON)
```

Out[29]:
```
{'decision_id': 'ba292a4f0d87dec4',
 'selected': 'ensemble_mean',
 'test_revealed': False,
 'reused': False}
```

Final reveal: five untouched runs

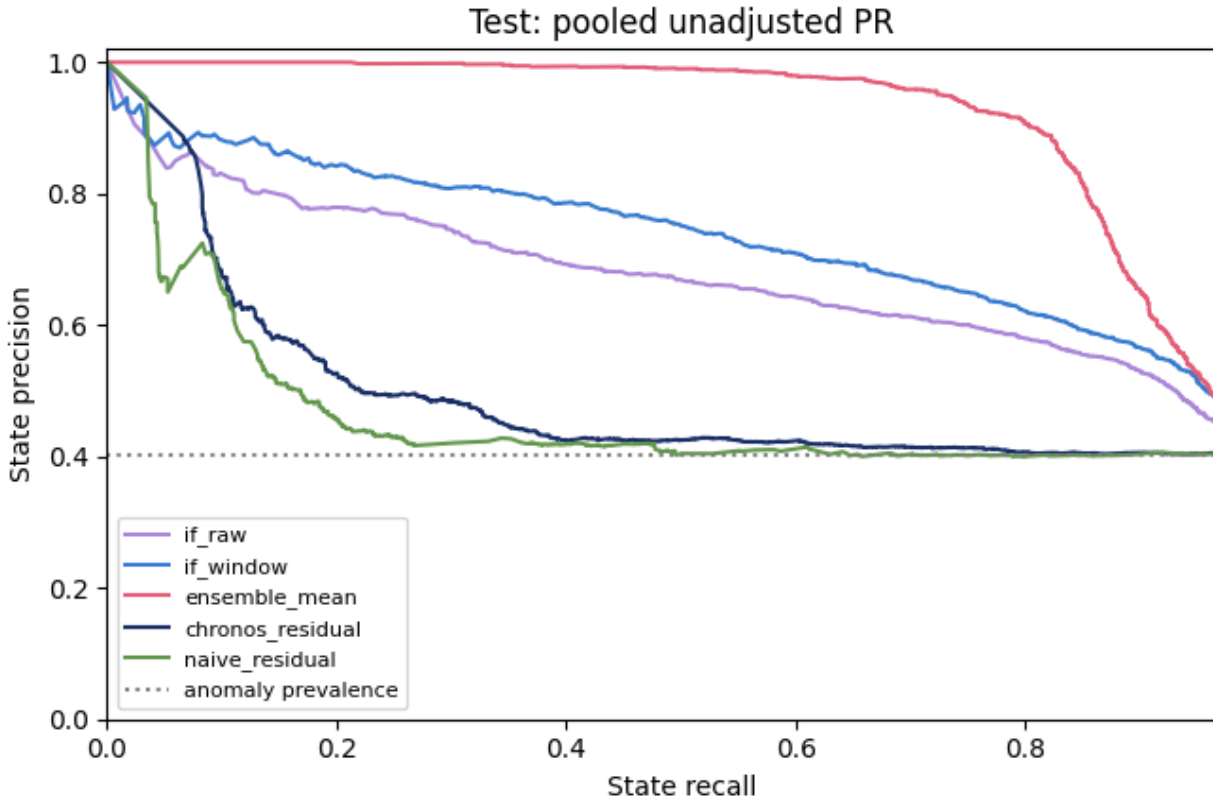
Your fitting data, CDF references, method and thresholds are now frozen. The next cell scores th held-out runs once. A repeated viewer reuses the saved result. Explain a disappointing result; do return to validation selection or revise weights after seeing this table.

In [30]:
```
holdout_report = SESSION.reveal_holdout()
compact_report(holdout_report)
```

Out[30]:

	detector	AP	state F 1	coverage	normal alarm fraction	onsets ≤ 6 0 s
0	robust	0.8237	0.0000	0.0000	0.0000	0
1	pca_q	0.9245	0.0000	0.0000	0.0000	0
2	if_raw	0.6746	0.0704	0.0371	0.0034	2
3	if_window	0.7336	0.0361	0.0186	0.0010	3
4	ensemble_mean	0.9206	0.5144	0.3575	0.0014	4
5	ensemble_max	0.8353	0.0000	0.0000	0.0000	0
6	chronos_residual	0.4900	0.1482	0.0821	0.0128	5
7	chronos_interval	0.4393	0.0029	0.0015	0.0000	0
8	naive_residual	0.4646	0.0784	0.0420	0.0091	4

In [31]:
```
SESSION.plot_pr('test', ['if_raw', 'if_window', 'ensemble_mean', 'chronos
```



In [32]:
```
FROZEN_CHOSEN = SESSION.decision['selected']
selected_runs = core.evaluate_runs(SESSION.rows[SESSION.rows.split == 'te
selected_runs[['run_id', 'ap', 'f1', 'state_coverage_mean', 'normal_alarm
```

Out[32]:

	run_id	ap	f1	state_coverage_mean	normal_al
0	valve 1 / 1 2	0.992004	0.541133	0.370927	0.0
1	valve 1 / 1 3	0.914317	0.450485	0.290727	0.0
2	valve 1 / 1 4	0.880923	0.312896	0.185464	0.0
3	valve 1 / 1 5	0.904867	0.564831	0.393564	0.0
4	valve 2 / 3	0.911110	0.702439	0.546835	0.0

Explain the decision, not just the leaderboard

Answer with exact runs/intervals and measured numbers.

1. Did history help the same IF? Name a benefit and a cost visible in a timeline.
2. Did classical fusion improve over the best singleton? What disagreement explains it?
3. How did Chronos compare with matched naive residuals and with classical state detectors?
4. Did the validation choice transfer? State AP, F1, coverage and false-alarm evidence.
5. What would have to be tested before using this detector on another pump/fault family?

A classical method, an ensemble or the foundation model may win. The task does not require a predetermined ordering. This small same-rig experiment cannot establish that one model family generally superior.

In [33]:
```
ANSWERS = {
    'representation': (
        'if_window против if_raw на валидации: AP вырос с 0.6730 до 0.767
        'покрытие — с 0.0570 до 0.1648 при почти равном бюджете ложных (0
        'окна 30/120 с добавляют историю, которая ловит медленный дрейф т
        'записи valve1/8: на временном графике (SESSION.plot_timeline) ви
        'уводят целые протяжённые участки аномалии от нормального кластер
        'всплески. На тесте преимущество окна сокращается (F1 0.0704 прот
        'надёжнее ранжировать, но не всегда шире тревожить.'),
    'fusion': (
        'ensemble_mean превосходит каждого участника на обеих выборках: в
        '0.2615 (if_window), 0.1044 (if_raw), 0 (robust/pca_q/ensemble_ma
        'F1=0.5144 при AP=0.9206. Конкретное расхождение источников видно
        'для valve1/8: там, где robust один держится ниже порога во время
        'каналов, поднимается pca_q; усреднение ECDF-рангов сохраняет тре
        'источник «выпадает». Важно: ensemble_max имеет AP=0.8353 на тест
        'резко наказывает шум одного источника и уходит в тихую политику.
    'forecasting': (
        'Прогнозное качество на валидации: Chronos снижает NMAE с 0.7981
        '(относительное улучшение 25.2%), причём на аномальном режиме 0.6
        'детектор состояния chronos_residual слабее классической фузии: в
        'AP=0.4900/F1=0.1482 против ensemble_mean 0.9206/0.5144. Остаток
        '≤60 с на тесте) и замолкает в середине аварии, когда новый режим
        'naive_residual шумнее (тест AP=0.4646/F1=0.0784). Вывод: базовая
        'временных рядов, но в этом протоколе проигрывает детекторам сост
    'transfer': (
        'Зафиксированный выбор — ensemble_mean (заблокирован на валидации
        'выбора не было). Точные метрики на пяти нетронутых тестовых запи
        'покрытие=0.3575, доля ложных тревог на нормальных=0.0014, 4 нача
        'Ближайший преследователь по AP на тесте — pca_q (0.9245), но он
```

```
        'основной метрике состояния ансамбль выигрывает с большим отрывом
        'подтверждено: валидация F1=0.5207 -> тест F1=0.5144 без деградац
    'scope': (
        'Ограничения: пороги настроены на 4 валидационных записях при бюд
        'ECDF-ранги насыщаются выше максимальной калибровочной оценки (см
        'поощряет удержание тревоги и смещает к широким интервалам; прогн
        'зафиксированного кэша, а не вычислены вживую (RUN_LIVE_CHRONOS=F
        'записям SKAB насоса/клапана с сеткой 1 Гц и этому протоколу. Сле
        'пер-записевые доверительные границы доли ложных тревог (bootstra
        'ускоряет ли раннее обнаружение добавление скользящих статистик в
}
ANSWERS
```

Out[33]:
```
{'representation': 'if_window против if_raw на валидации: AP вырос с 0.6730
0.7679, state F1 — с 0.1044 до 0.2615, покрытие — с 0.0570 до 0.1648 при поч
равном бюджете ложных (0.0093 против 0.0090). Каузальные окна 30/120 с добав
историю, которая ловит медленный дрейф тока двигателя и RMS вибрации в запис
valve1/8: на временном графике (SESSION.plot_timeline) видно, что скользящие
статистики уводят целые протяжённые участки аномалии от нормального кластера
только изолированные всплески. На тесте преимущество окна сокращается (F1 0.
против 0.0361) — история помогает надёжнее ранжировать, но не всегда шире
тревожить.',
 'fusion': 'ensemble_mean превосходит каждого участника на обеих выборках:
валидация F1=0.5207 против 0.2615 (if_window), 0.1044 (if_raw), 0 (robust/pc
ensemble_max из-за тихих порогов); тест F1=0.5144 при AP=0.9206. Конкретное
расхождение источников видно на SESSION.plot_complementarity() для valve1/8:
где robust один держится ниже порога во время дрейфа, меняющего взаимосвязи
каналов, поднимается pca_q; усреднение ECDF-рангов сохраняет тревогу, когда
одиночный источник «выпадает». Важно: ensemble_max имеет AP=0.8353 на тесте,
F1=0 — максимум слишком резко наказывает шум одного источника и уходит в ти
политику.',
 'forecasting': 'Прогнозное качество на валидации: Chronos снижает NMAE с 0.
(persistence) до 0.5968 (относительное улучшение 25.2%), причём на аномальнс
режиме 0.6340 против 0.8315 (23.8%). Но как детектор состояния chronos_resic
слабее классической фузии: валидация AP=0.4416/F1=0.0909, тест AP=0.4900/F1=
против ensemble_mean 0.9206/0.5144. Остаток выигрывает на переходах (5 собы
с на тесте) и замолкает в середине аварии, когда новый режим попадает в 128-
контекст; naive_residual шумнее (тест AP=0.4646/F1=0.0784). Вывод: базовая м
полезна как предиктор временных рядов, но в этом протоколе проигрывает детек
состояния по поддержке длинных интервалов.',
 'transfer': 'Зафиксированный выбор — ensemble_mean (заблокирован на валидац
reveal_holdout(), повторного выбора не было). Точные метрики на пяти нетрону
тестовых записях: AP=0.9206, state F1=0.5144, покрытие=0.3575, доля ложных т
на нормальных=0.0014, 4 начала аварий обнаружены за <=60 с. Ближайший
преследователь по AP на тесте — pca_q (0.9245), но он «тихий» (F1=0), поэтом
основной метрике состояния ансамбль выигрывает с большим отрывом; обобщение
новые записи подтверждено: валидация F1=0.5207 -> тест F1=0.5144 без деграда
 'scope': 'Ограничения: пороги настроены на 4 валидационных записях при бюдж
~1%, оценка хвоста шумная; ECDF-ранги насыщаются выше максимальной калибровс
оценки (см. столбец saturation); state-F1 поощряет удержание тревоги и смеща
широким интервалам; прогнозные признаки взяты из зафиксированного кэша, а не
вычислены вживую (RUN_LIVE_CHRONOS=False); выводы относятся к этим записям S
насоса/клапана с сеткой 1 Гц и этому протоколу. Следующий проверочный экспер
пер-записевые доверительные границы доли ложных тревог (bootstrap по нормаль
интервалам) и проверка, ускоряет ли раннее обнаружение добавление скользящих
статистик в окно прогнозного остатка.'}
```

In [34]:
```
SESSION.save_submission(ANSWERS)
```

Out[34]:
```
{'decision': '/workspace/work/skab_lab_runs/5a53a43f10e54ca880f1a67bb9c6371
frozen_decision.json',
 'metrics': '/workspace/work/skab_lab_runs/5a53a43f10e54ca880f1a67bb9c63717/
holdout_metrics.csv',
 'interpretations': '/workspace/work/skab_lab_runs/
5a53a43f10e54ca880f1a67bb9c63717/interpretations.json'}
```

Optional: run one live foundation forecast

This extension is **off by default** and is not needed for the assessed route. Enabling it downloads approximately 1 1 2 MB of weights and adds Torch/Chronos dependencies. Use a fresh installation conflicts arise. CPU is supported; optional GPU changes speed, not the experiment c The recorded cache remains identified separately from this new one-block execution.

In [35]:
```
RUN_LIVE_CHRONOS = False
if RUN_LIVE_CHRONOS:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q',
    import torch
    from chronos import Chronos2Pipeline
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    live_pipeline = Chronos2Pipeline.from_pretrained('autogluon/chronos-2
    live_frame = SESSION.frames['valve1/8'].iloc[:128]
    live_input = live_frame[CHANNELS].to_numpy(dtype=np.float32).T
    live_output = live_pipeline.predict([{'target': live_input}], predict
    print({'new_execution': True, 'device': device, 'prediction_shape': t
else:
    print('Optional live model execution is off. The required route used
```

```
Optional live model execution is off. The required route used the labelled re
forecast cache.
```

Submit and attribute

Save your completed notebook and the three small files listed above. Do not submit model cach credentials. Be prepared to explain or modify every task function.

Checks: four task contracts pass; fresh top-to-bottom run works; split identities stay fixed; no fut leakage; frozen decision precedes heldout reveal; five answers cite actual evidence. Teacher so remain in the separate private notebook.

Sources: SKAB repository, Katser & Kozitsin (2 0 2 0), dataset DOI, GPL- 3 . 0 ; Chrono small model, Chronos implementation. The exact dataset/model revisions and recorded executic environment are included in the verified package. No claim of general foundation-model superio inferiority follows from this one laboratory.